Making the UREX Tread-Depth Software Robust: A Benchmark Study of Ten Algorithm Families on Simulated and Real Laser-Line Images

UREX project team

Technical report · 12 November 2026 · code: `github.com/Advait-B/urex`, branches `research/*`

1 Introduction

SMRT inspects more than 1,000 buses, and each inspection includes measuring tyre tread depth by hand at several points on every tyre. The UREX device replaces the hand gauge with a handheld optical head: a camera looks straight down at the tread and a green line laser strikes the tread at an angle. Where the rubber is lower (inside a groove) the laser line appears shifted in the image, and the size of the shift gives the depth. This is *laser triangulation*.

The software has one job: turn one greyscale image into a groove depth in millimetres, and a PASS / FAIL against the 1.6 mm legal limit. The pipeline we had on 12 November (the *baseline*) does this in three steps: (1) find the laser line with a Steger ridge detector, (2) fit the tread surface and read how far the line drops inside each groove, (3) convert pixels to millimetres with a calibration. It works on good photos, but it also reports a depth on most *bad* photos, and it has never been tested against known answers.

This report asks a practical question: **which algorithms actually make the software more robust, and by how much?** To answer it we built a simulator of the printed holder, a fixed benchmark with exact ground truth, and then tried ten families of algorithms, each on its own git branch, all scored by the same harness on the same frames and on the 86 real photos taken between 8 and 30 October.

Contributions.

- A ray-traced simulator of the Rev A holder on a tyre, with exact truth for the laser line and every groove (Section 3.1).
- A benchmark of five simulated test sets plus the real photos, and a scorer that rewards an algorithm for saying "retake the photo" instead of guessing (Section 3.2).
- A like-for-like evaluation of ten algorithm families: sub-pixel peak detectors, dynamic-programming line tracking, a small U-Net, a candidate classifier, a robust depth stage, calibration models, a quality gate, laser on/off subtraction, an end-to-end network, and a combined stack (Sections 4–**??**).
- An independent audit of the numbers, and an honest list of what the simulator and the real photos can and cannot tell us (Section **??**).

2 The device and the measurement

Figure 1 shows the geometry of the printed holder. The holder's feet rest on the tread, so the camera height and the laser angle are fixed. The numbers used throughout are in Table 1.

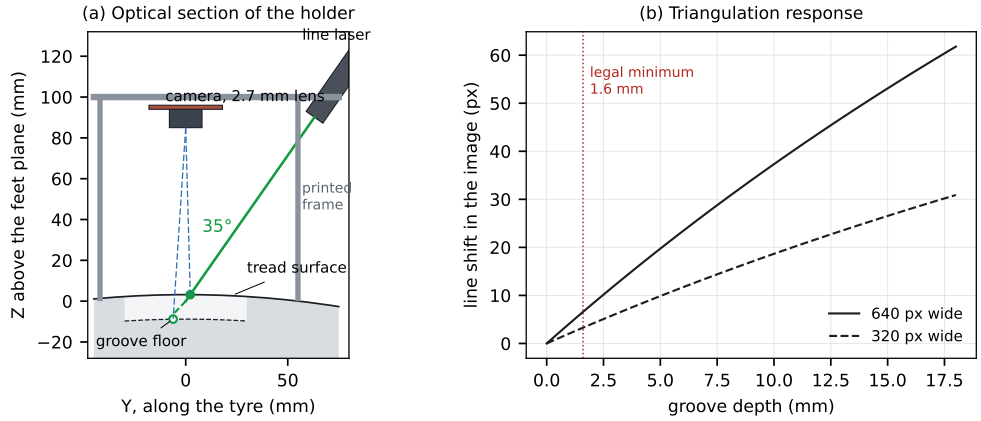


Figure 1: Side view of the holder on a tyre. The camera looks straight down; the laser sheet arrives at 35°. A groove floor lies lower than the tread, so the laser meets it further along, and the camera sees the line shifted.

Table 1: Device constants (holder Rev A, OV5640 camera with a 2.7 mm lens).

Quantity	Value	Comment
Camera height above the feet plane	85 mm	lens front, looking straight down
Focal length	2.7 mm (235 px at 320 px image width)	horizontal field of view 68°
Laser angle θ	35° from vertical	110° fan across the tread
Tyre radius / crown height	480 mm / 3.2 mm above the feet plane	bus tyre between the feet
Image used by the algorithms	320 × 240 greyscale	down-sampled; real photos are 240 × 320
Legal limit	1.6 mm	PASS only if every groove is deeper

How a depth becomes a pixel shift. Let H be the distance from the lens to the tread surface (about 82 mm) and f the focal length in pixels. A groove of depth d moves the laser spot sideways by $d \tan\theta$ on the tyre and puts it d further from the camera, so the line moves in the image by

$$\Delta v \;=\; \frac{f\, d \tan\theta}{H + d} \qquad \Longrightarrow \qquad d \;=\; \frac{H\,\Delta v}{f \tan\theta - \Delta v}. \tag{1}$$

With our numbers this is about **2.1 px of shift per millimetre of depth**, and about 2.9 px per millimetre across the tread. Two consequences follow, and they explain most of this report:

- An error of only 0.2 px in locating the line is already 0.1 mm of depth. Sub-pixel accuracy matters.
- One wrong decision about *which bright thing is the laser* (a glint, a wet patch, the edge of a thick saturated line) is an error of many millimetres. Robustness matters more.

In practice we do not use Eq. (1) directly. A calibration (Section 4.6) maps each image position to a height, which also absorbs lens distortion and assembly tolerances.

3 Method: simulator, benchmark and scorer

3.1 Simulator

Real photos have no ground truth: nobody measured those grooves with a gauge. So we wrote a simulator, `research/sim/holder_sim.py`, that renders what the camera in the holder would see. It builds a tyre surface (radius 480 mm, a gentle crown, circumferential grooves with sloped walls and optional rounded floors), traces 2,400 laser rays to the first surface they hit, checks that the camera can see each hit, and renders the line with realistic blur, sensor noise, lens distortion and JPEG compression. Because the scene is known, every frame comes with the exact sub-pixel laser line and the exact depth of every groove, plus a laser-off frame of the same scene.

The simulator can also make conditions worse on purpose: rocking of the holder, sunlight, wet glints, stones in grooves, zig-zag grooves, mud that hides part of the line, paint, a dim or a saturated laser. Figure 2 compares simulated frames with real photos.

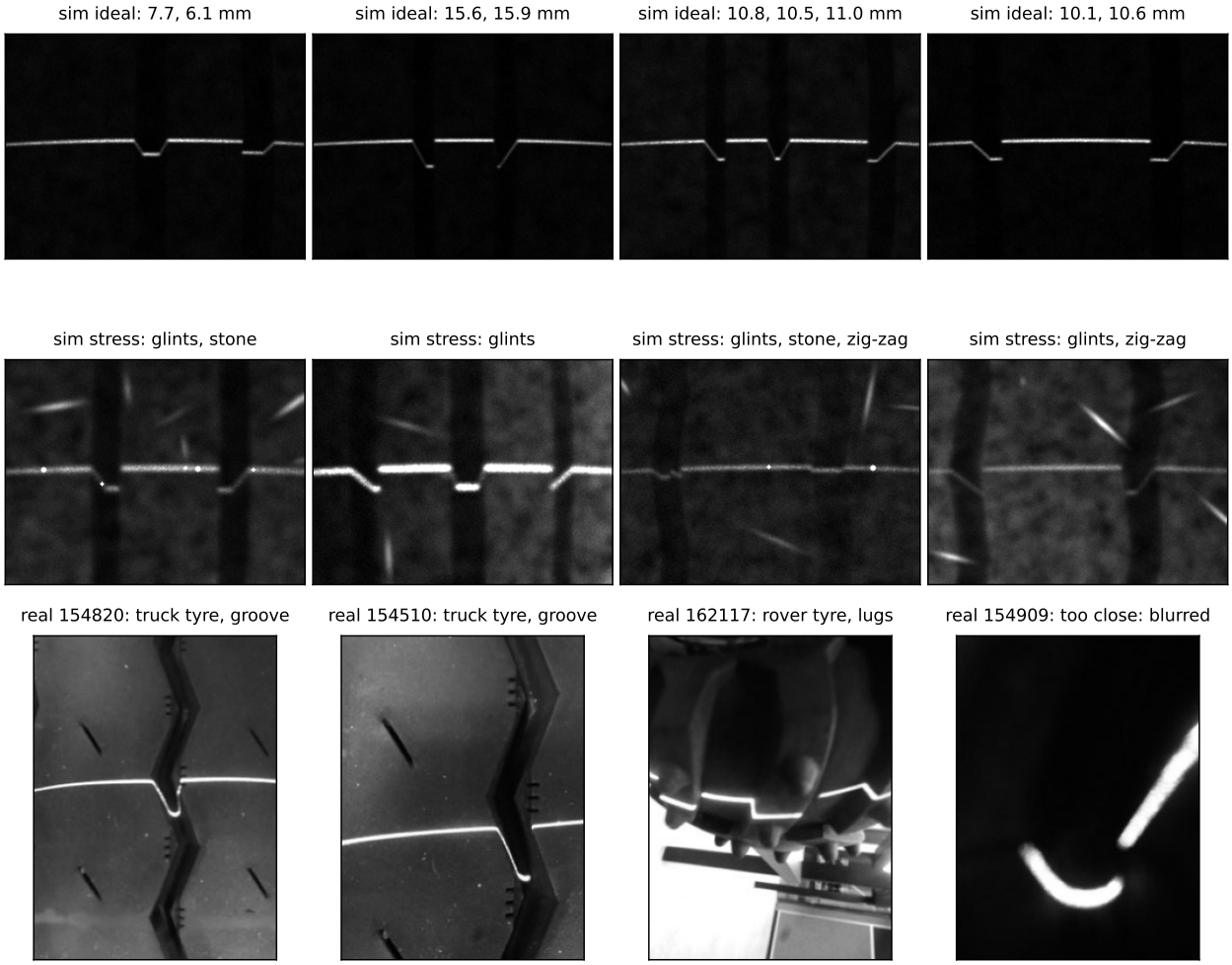


Figure 2: Simulated frames (holder locked in place) next to real photos (hand-held, October). The simulated "ideal" line is thin and level; real lines are thicker, often saturated, and can sit anywhere in the frame.

3.2 Test sets

Table 2: The benchmark. Algorithms may train or tune only on `train`; all other sets are for testing.

Set	Frames	What it represents
`ideal`	150	Holder locked on a clean tyre in shade: the conditions the holder is designed to create.
`stress`	200	Rocking $\pm 1^\circ$, sunlight, wet glints, stones, zig-zag grooves, mud gaps, paint, blur, dim or saturated laser.
`legal`	150	Worn tyres, grooves 0.6–3.5 mm: the PASS/FAIL boundary at 1.6 mm.
`shifted`	150	Three *other* holder units (laser at 33°, 36°, 37°; small roll and height errors), each with its own calibration. Tests whether a method survives a different unit.
`handheld`	150	Built to look like the real photos: thick saturated line, line anywhere in the frame, clutter. Scored in pixels.
`train`	1,200	Mixed conditions, different random seed. Training only.
`calib`	15	Flat plate at 15 known heights from $+6$ to -22 mm.
real photos	86	Hand-held photos, 8–30 October. A person would measure 19 of them. Six truck-tyre frames have a reference groove depth in pixels.

3.3 What is scored

Every algorithm is a Python module with the same interface: image in, list of grooves out (plus, optionally, the laser line it found and a number saying whether the frame is usable). The scorer reports:

- **Coverage**: share of frames the algorithm answers. An algorithm may *abstain* ("retake").
- **Depth MAE, median, P95**: mean, median and 95th-percentile absolute depth error over grooves that were found, in mm. The mean is dominated by a few huge errors; the median shows the typical case.
- **Gross**: share of grooves wrong by more than 1 mm (3 px on `handheld`).
- **Miss**: measurable grooves that were not reported. **False**: reported grooves per frame where there is none.
- **Verdict**: PASS/FAIL accuracy at 1.6 mm over answered frames. **F.pass**: unsafe mistakes (the tyre is illegal and the algorithm says PASS), counted over *all* frames.
- **CPU ms**: processor time per frame on one laptop-class core.

For real photos there is no ground truth, so the scorer reports agreement instead: agreement of the line with a reference line on the six truck-tyre frames, the difference from the reference depth in pixels, how many of the 19 good photos are kept, and on how many of the 67 bad photos a depth is (wrongly) reported.

Caution. The real-photo references were produced by the baseline pipeline and then checked by eye. The baseline therefore scores "perfect" on them by construction. On real photos, read the numbers as *agreement with the baseline on frames where the baseline looked right*, not as accuracy.

4 Algorithms and results, one family at a time

Each family lives on its own branch (Table 3). Unless stated otherwise, a line extractor is paired with the benchmark's default depth stage, so differences come from the line alone. Full comparison tables for all sets are in Section **??**.

Table 3: Branches. All branch from `research/base` (simulator + benchmark) and were merged into `research/integration` for re-scoring.

Branch	Stage it changes	Uses learning?
`research/peak-detectors`	finding the line (per image column)	no
`research/dp-tracking`	finding the line (global path)	no
`research/cnn-segmentation`	finding the line	yes, U-Net (85 k weights)
`research/pixel-classifier`	choosing which candidates are laser	yes, gradient boosting
`research/robust-depth`	line → groove depth	no
`research/calibration`	pixels → millimetres	no
`research/quality-gate`	"is this photo usable?"	yes, logistic regression
`research/onoff-fusion`	two photos: laser on, laser off	no
`research/end-to-end`	image → depth directly	yes, CNN (155 k weights)
`research/best-stack`	combination of the above	(gate only)

4.1 Sub-pixel peak detectors

Idea. The laser line runs left to right, so in every image column there is one brightness peak. Find it and refine its position below one pixel. This is the classic approach in laser scanners and is far cheaper than the baseline's 2-D ridge detector. We compared centre of gravity, a 3-point Gaussian fit, the Blais–Rioux filters, and a smoothed peak with a parabola fit; all after removing the background with a morphological top-hat.

```
def parabola3(W, off):                    # W: (N, 3) response samples around each column peak
    h = len(off) // 2
    a, b, c = W[:, h - 1], W[:, h], W[:, h + 1]
    den = a - 2 * b + c
    d = np.where(den < -1e-6, 0.5 * (a - c) / np.where(den < -1e-6, den, -1.0), 0.0)
    return np.clip(d, -1.0, 1.0)

resp = smooth(img, 1.5, 1.5)              # img = top-hat of the frame
thr = max(12.0, 0.25 * np.median(resp.max(axis=0)))
iv, iu = column_peaks(resp, thr, max_per_col=3)
v = iv + parabola3(*windows(resp, iv, iu, 1))
```

Listing 1: Smoothed peak + parabola (`algos/pk_fir2d.py`). The parabola through the three samples around the maximum gives the sub-pixel row.

Results. In clean conditions every per-column detector beats the baseline: depth MAE on `ideal` falls from 0.147 to about 0.06 mm, and line error from 0.16 to 0.11–0.12 px. They are also the cheapest methods (3–5 ms of their own CPU time against 11 ms for Steger). Under stress the *typical* error is excellent (median 0.09 mm against 0.54 mm), but the mean stays near 1 mm, because a per-column detector has no idea of a *line*: one glint that is brighter than the laser in a few columns becomes a false groove. The 3-point Gaussian fit collapses when the line saturates (verdict accuracy 0.83 on `stress`). On real photos only the variant with an extra pass along the rows (`pk_fir2d_rows`, which recovers steep groove walls) finds all six truck grooves.

Takeaway. Per-column peaks are the most accurate and cheapest way to *locate* a line you have already identified. They are not enough on their own to decide *which* bright pixels are the line.

4.2 Dynamic-programming line tracking

Idea. Use the fact that the laser line is one connected curve crossing the image. Give every pixel a "reward" (how line-like it is) and find the single left-to-right path with the best total reward, paying a small price for gentle slopes and a fixed price for a jump (a groove edge). This is the Viterbi algorithm; it is exact, needs no training, and one glint cannot win because the path would have to pay to jump to it and back.

```
# pen(d) = A*|d|       for |d| <= s   (gentle slope, e.g. a lit wall: cheap)
#        = J + B*|d|   otherwise      (a groove edge: fixed price + a little per px)
for c in range(1, W):
    full = C[c - 1]                                  # C[c, v] = cost of the best path ending at (c, v)
    prev = full[s:s + H]
    near = prev.copy()
    for k in range(1, s + 1):                        # smooth move of k rows up or down: A per px
        np.minimum(near, full[s - k:s - k + H] + A * k, out=near)
        np.minimum(near, full[s + k:s + k + H] + A * k, out=near)
    # jump from any row v': prev[v'] + J + B*|v - v'|, as two running minima (from above, from below)
    far = J + np.minimum(acc(prev - Br) + Br, acc((prev + Br)[::-1])[::-1] - Br)
    C[c, s:s + H] = np.minimum(near, far) - R[c]      # R = reward in 0..1
```

Listing 2: Core of the Viterbi pass (`algos/dp_dense.py`): cost of the best path ending at each pixel, column by column.

Results. `dp_dense` is the best single-frame, training-free line extractor in the study (Figure 3). On `stress` the line F1 rises from 0.82 to 0.94, missed grooves fall from 9.9 % to 1.8 %, depth MAE from 1.52 to 0.54 mm and P95 from 6.7 to 0.59 mm, with no unsafe PASS. On `handheld` it finds 96 % of grooves where the baseline finds 53 %. The work on this branch also explained two baseline faults: Steger's ± 0.5 px acceptance test leaves holes that split one groove into two, and on a saturated line Steger marks the two *edges* of the bright band instead of its centre. DP takes the mid-point of the band at half maximum instead. Cost: about $1.4\times$ the baseline's run time.

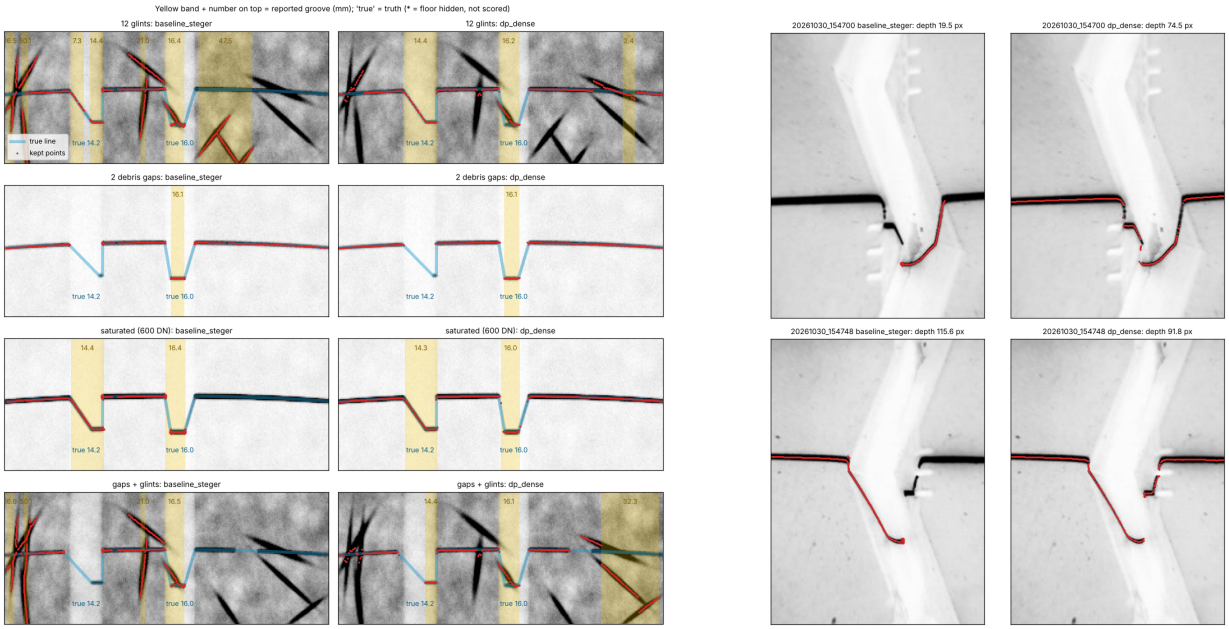


Figure 3: Left: baseline (left column) against DP tracking (right column) on the same simulated scene with one difficulty switched on at a time; yellow bands are reported grooves. Right: two real photos with a thick saturated line; the baseline (left) follows an edge of the band, DP (right) follows its centre.

What still fails. Frames with many glints still produce about 0.1 false grooves per frame under stress, and paint or mud that hides the laser cannot be tracked through. On real photos DP agrees with the reference to 1.1 px (five of six within 3 px).

Takeaway. A global "one line" constraint removes most gross errors and costs no training data. This is the line extractor we recommend.

4.3 Learned line detection: a small U-Net

Idea. Train a neural network to paint a heat-map of where the laser line is, then read the peak in each column. The network is a three-level U-Net with 84,705 weights (0.36 MB), trained only on the 1,200 simulated `train` frames with a soft cross-entropy plus Dice loss.

Results. Trained without augmentation, the network gives the best single-frame numbers on simulated stress (MAE 0.236 mm, P95 0.47 mm, verdict 0.985) but fails on real photos: it disagrees with the reference by 7 px. The cause is that every simulated line sits at the same height in the frame and has the same thickness; the network memorised that. With augmentation (random shifts, rotation, thickness and gain) real agreement improves to 1.1 px and `handheld` line F1 reaches 0.90, but stress MAE rises to 0.67 mm. It still needs a hand-written "keep one line" rule after the heat-map, and at 60–90 ms per frame on a laptop core it would take seconds on the STM32H7.

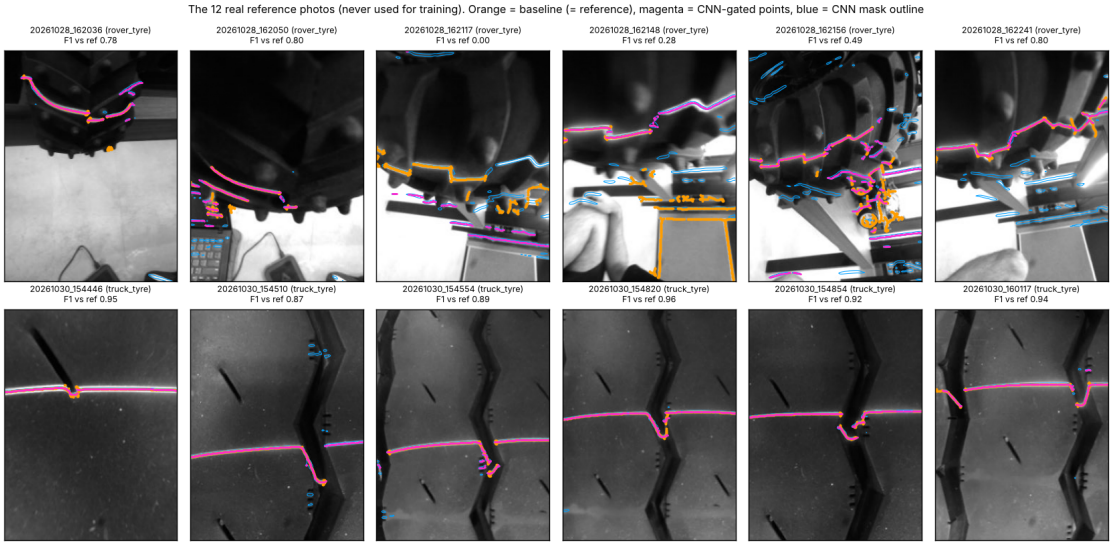


Figure 4: The augmented U-Net on the twelve real reference photos, which it never saw in training (orange: reference, magenta: network). It follows the line on the truck tyre (bottom row) but is confused by background clutter on the rover tyre (top row).

Takeaway. A network trained only on simulation reaches roughly the same real-photo agreement as DP tracking at more than twice the cost. It becomes worth revisiting once there are a few hundred *real* holder images to train on.

4.4 Candidate classifier

Idea. Keep the classical detector, but let a small learned model decide which of its candidate points are laser and which are glints. Each candidate gets 30 hand-made features (brightness, width, how well it fits its neighbours, distance from the most common row) and a gradient-boosted classifier (scikit-learn `HistGradientBoostingClassifier`, 487 kB) keeps those with probability ≥ 0.95.

Results. On simulated stress this is very strong: MAE 0.273 mm, false grooves 0.02 per frame, verdict 0.970. But the feature-importance plot (Figure 5) shows it relies almost entirely on one feature, the distance from the most common row, which works because the simulated line always sits at row 115 ± 1. On real photos it found two of the six reference grooves. A version without that feature is honest but weaker.

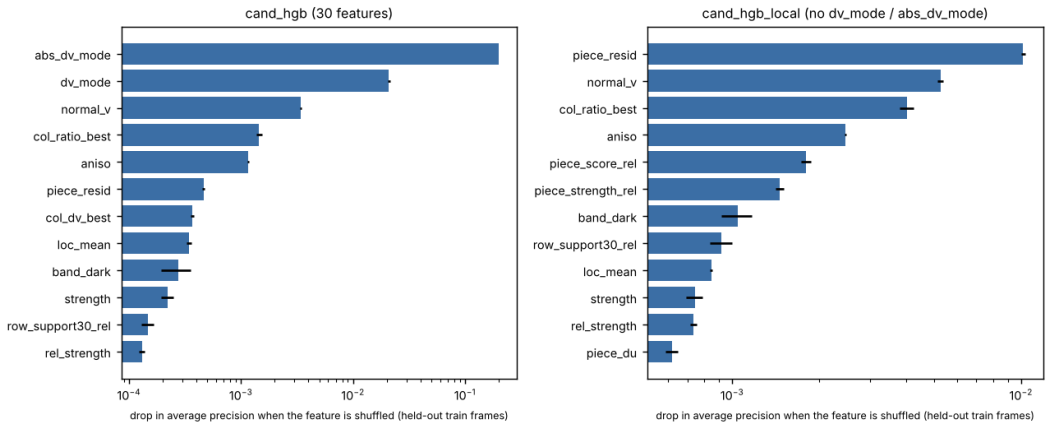


Figure 5: What the candidate classifier relies on. Left: the full model leans on "distance from the most common row" (`abs_dv_mode`), a shortcut that only exists in the simulator. Right: without it, the model uses local fit quality instead.

Takeaway. A good score on simulation can come from a shortcut. Looking at what the model uses, and testing on real photos, exposed it. This result is the reason the `handheld` and `shifted` sets were added.

4.5 Robust depth stage

Idea. Suppose the line is perfect. How wrong is the depth? With the default stage, about 0.07 mm on average and too deep. This branch studied the second half of the pipeline (line → depth) on its own and rebuilt it in five small steps:

1. Convert every line point to a height and to its true sideways position on the tread, and fit the tread surface in millimetres rather than in image columns.
2. Fit the surface with RANSAC (a quadratic). Grooves are one-sided outliers, and the alternatives we tried (Huber, Theil–Sen) break when grooves fill much of the view.
3. Detect grooves, including shallow ones, with a running median, and drop narrow shallow blips.
4. Take the groove floor as the deepest cluster found by DBSCAN, and its median as the depth. A single stray point can no longer set the depth.
5. Subtract a small tyre-curvature term, and attach an uncertainty and a "floor not fully visible" flag.

```
def lateral_px(u, v, z, width=320, height=240):
    f = F_PX * width / 320.0
    xd, yd = (u - (width - 1) / 2) / f, (v - (height - 1) / 2) / f
    x, y = xd.copy(), yd.copy()
    for _ in range(3):                          # undo the radial distortion
        k = 1 + K1 * (x * x + y * y)
        x, y = xd / k, yd / k
    return 160.0 + 2.9 * x * (ZC_MM - z)        # X in mm, expressed as "ideal pixels"

# in stage():   u = lateral_px(px, v, -q);  coef = RANSAC quadratic of q against u;  r = q - polyval(coef, u)
# per groove:   z_s = -polyval(coef, u_floor);  z_f = z_s - d_raw
#               d = (z_s - z_f) - TAN_LASER**2 * (z_f**2 - z_s**2) / (2 * tyre_R)
```

Listing 3: Step 1 (`algos/robust_depth.py`): undo lens distortion and express each point's sideways position in mm before fitting the tread surface.

Results. With a perfect line, depth MAE on `ideal` falls from 0.071 to 0.017 mm and the bias from +0.068 to −0.012 mm (Figure 6, top). Most of the old bias was *not* tyre curvature, as we first believed: it came from camera parallax and lens distortion bending the crown in image columns. The curvature term itself only matters for deep grooves (+0.09 mm at 17 mm) and is insensitive to the assumed tyre radius (±0.005 mm for ±20 %).

With the baseline line, the new stage (`rd_final`) cuts stress MAE from 1.52 to 0.46 mm and, on worn tyres, MAE from 0.053 to 0.019 mm with *no missed grooves*. The shallow-groove rule matters for safety: on a special set of 300 unevenly worn tyres the default stage called 9 % of illegal tyres PASS because it did not see the shallowest groove at all; the new stage called none (Figure 6, bottom).

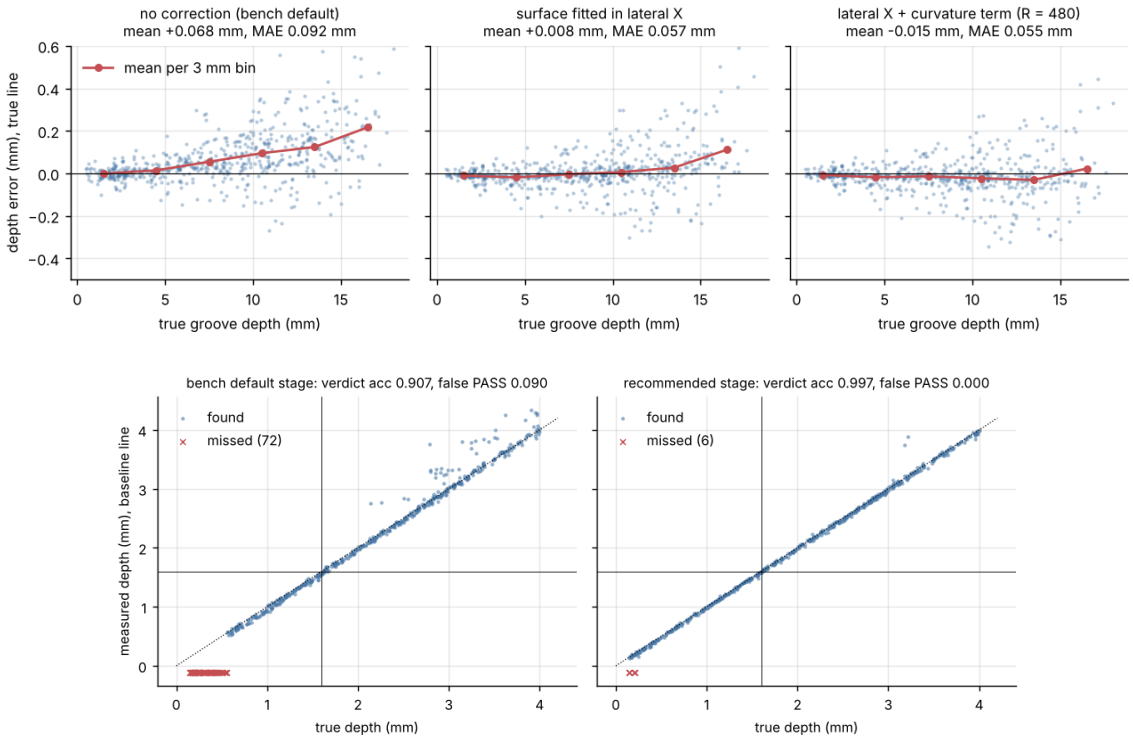


Figure 6: Top: depth error with a perfect line against true depth: default stage, surface fitted in millimetres, and with the curvature term added. Bottom: measured against true depth on unevenly worn tyres; red crosses are grooves the stage did not see. The vertical and horizontal lines mark 1.6 mm.

Limits. Rocking the holder about the tread axis adds about 1.2 % of the depth per degree, and one laser line cannot detect it. The correction uses holder constants (camera height, lens distortion), and the simulated crown is an exact parabola, so the gain on real treads will be smaller than in simulation. On real photos the DBSCAN floor disagrees with two of the six references by about 2.4 px; without a gauge reading we cannot say which is right, but on the one pair of repeated photos it is far more consistent (0.17 px apart instead of 2.45 px).

Takeaway. The depth stage was the largest source of error after gross line mistakes, and fixing it needs no learning: fit in millimetres, take the floor from a cluster, look for shallow grooves explicitly.

4.6 Calibration

Idea. Calibration turns "the line is at pixel (u, v)" into "the surface is at height z". The October procedure photographs a flat plate at known heights and fits a polynomial. We compared polynomials, splines, a Gaussian process, and a *physical* model with 5 or 7 parameters (pinhole camera, one distortion term, and the laser plane).

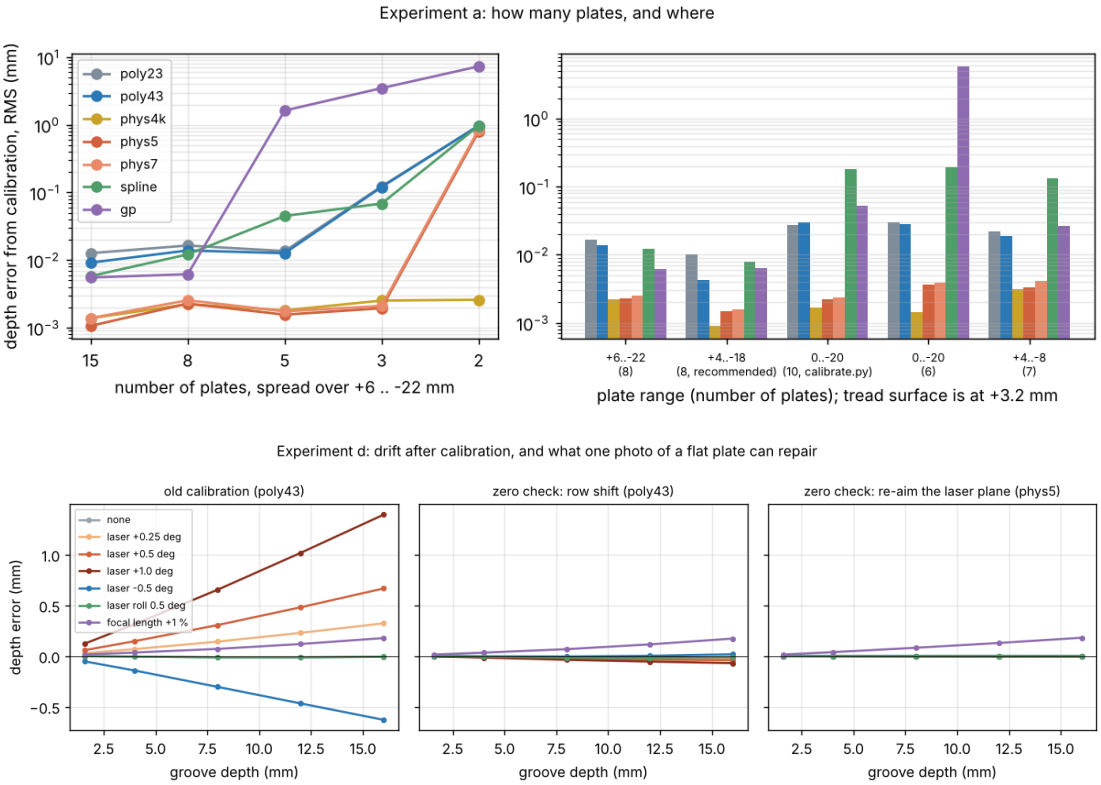


Figure 7: Top: calibration error against the number of plate heights (left) and their range (right); the physical models (red, orange) stay accurate with three plates. Bottom: depth error after the laser is knocked by a fraction of a degree (left), and after correcting with one photo of a flat plate at zero height (centre, right).

Results.

- With 15 plates every sensible model is accurate to better than 0.02 mm. With only three plates the physical model still gives 0.002 mm; the polynomial gives 0.12 mm.
- The October recipe (plates from 0 to -20 mm) never covers the tread itself, which sits at $+3.2$ mm, so the polynomial has to extrapolate. Recommended heights: $+4$, $+2$, 0, -3, -6, -10, -14, -18 mm.
- If the laser is knocked by 0.5°, every depth is scaled by about $+4\,\%$ (0.67 mm at 16 mm). One photo of a flat plate at zero height detects this, and re-aiming the laser plane in the physical model corrects it to within 0.003 mm. A change of focal length is *not* visible to that check.
- A systematic error in the shim heights passes straight through every model. Only a step of known depth can catch it.

Caution. The simulator's camera *is* a pinhole with one distortion term, so the physical model fits it perfectly by construction. Expect a smaller advantage on the real device; the ranking for few plates and for extrapolation should still hold.

Takeaway. Use the physical model, calibrate with eight heights that bracket the tread, and add a daily "zero check" on a flat plate plus a known-depth step block.

4.7 Quality gate

Idea. The baseline reports a depth on about 70 % of photos that a person would refuse to measure. A *gate* decides, before a number is shown, whether the photo is good enough. We built it in two stages: a *frame gate* (is there a laser line on a tyre at all?) trained on the 86 labelled real photos, and a *risk gate* (is this particular reading likely to be wrong?) trained on simulated frames where the error is known. Both are regularised logistic regressions on a few dozen hand-made features.

```
r = v - np.polyval(coef, u)                    # > 0 = below the fitted tread surface
z.update({
```

```
    "min_floor_pts": float(min(floor_n, default=0)),          # fewest points on any groove floor
    "depth_spread_px": float(np.ptp([x["depth"] for x in g])) if g else 0.0,   # grooves of one tyre wear alike
    "above_frac": float((r < -depth.PX["min_depth"]).mean()),  # points ABOVE the tread: glints, never rubber
    "max_gap_frac": float(np.max(np.diff(np.r_[0, uc, w - 1])) / w),          # biggest hole in the line
    "multi_frac": float((vr > 3).mean()),                      # columns where the line has two branches
})
```

Listing 4: Some of the risk-gate features (`algos/gate_features.py`). Each is a number a technician could also judge by eye.

Results. With the gate, the baseline answers 88 % of `ideal` frames, 96 % of `legal` frames and only 16 % of `stress` frames, but the answers it does give under stress are good: MAE falls from 1.52 to 0.14 mm and gross errors from 36 % to zero (Figure 8). On real photos it keeps 17 of the 19 usable frames and cuts false measurements from 69 % to 4 % on frames with no laser, and from 75 % to 25 % on frames where the laser is visible but unusable.

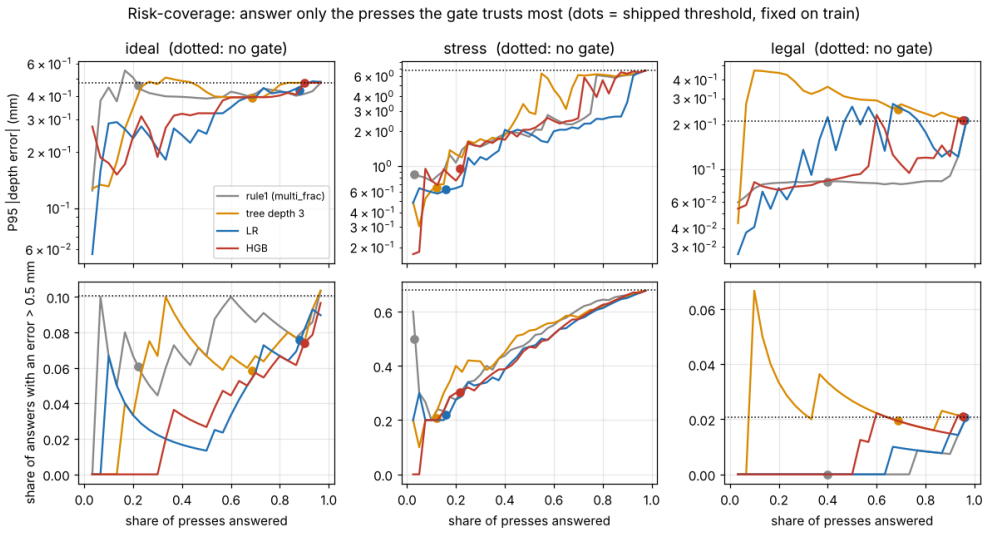


Figure 8: Risk–coverage curves: if the device only answers the share of presses it trusts most (horizontal axis), how bad are the answers (vertical axis)? Dots mark the threshold shipped, fixed on `train`.

Caution. The frame gate was trained on the same 86 real photos, so the real-photo numbers above are *in-sample*. The honest estimate is leave-one-session-out cross-validation: a false-accept rate of 0.24 (95 % interval 0.14–0.36) when every choice is nested inside the cross-validation. More complex models did not help: a small CNN gate was at chance on a new day (AUC 0.49).

Takeaway. "Retake the photo" is the cheapest robustness feature. With only 86 labelled photos, simple logistic regression is the right-sized model; more labelled sessions will help more than a bigger model.

4.8 Laser on/off subtraction

Idea. Take two photos a few milliseconds apart, one with the laser on and one with it off, and subtract. Everything that is not laser (sunlight, glints, paint) cancels. The October photos have no laser-off partner, so this was tested in simulation only, as a question to the hardware team: is it worth letting the microcontroller switch the laser?

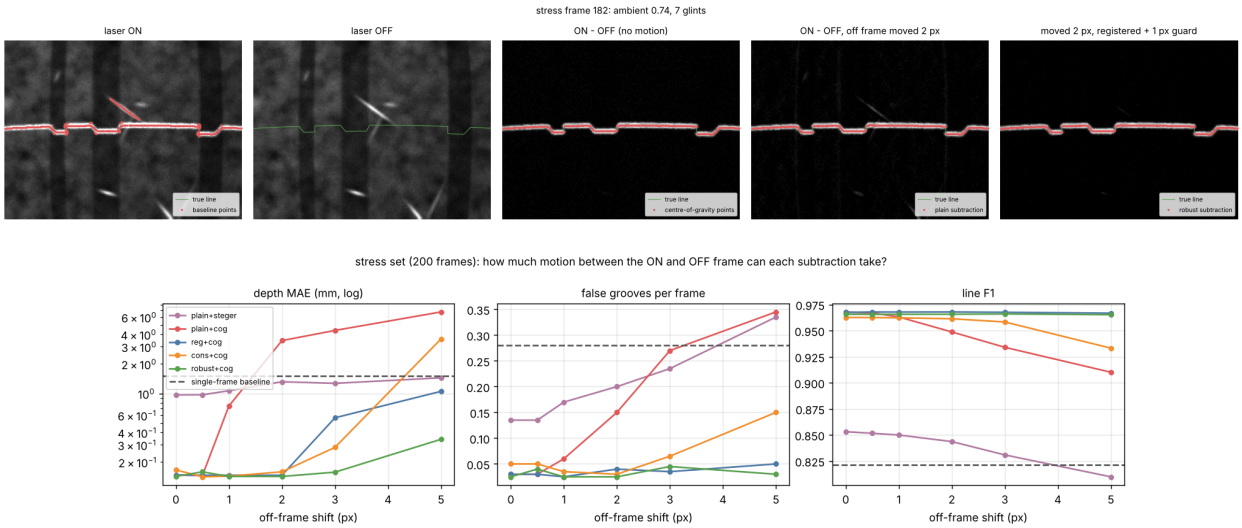


Figure 9: Top: a stress frame with glints; laser on, laser off, their difference, the difference when the hand moved 2 px between the photos, and the same after registration. Bottom: error against hand motion between the two photos, for plain and registered subtraction.

Results. With a simple centre-of-gravity detector on the difference image (`onoff_cog`), stress MAE is 0.147 mm, P95 0.40 mm, false grooves 0.01 per frame, and no unsafe PASS on that set: the best stress numbers of any single method. Motion matters: with 2 px of movement between the photos, plain subtraction is *worse* than a single frame. Matching the gain, registering the two frames by phase correlation and dilating the off frame by one pixel keeps the error flat up to 3 px of motion. Subtraction does not help with a saturating background, rocking, or a dim laser. Averaging several presses helps little, because the remaining error is mostly bias; a median of three is cheap insurance.

Takeaway. If the next board revision can switch the laser with a GPIO pin, on/off subtraction is the single biggest robustness gain available, and it is twenty lines of code.

4.9 End-to-end learning

Idea. Skip the geometry: train a network to output groove depths directly from the image. A CNN with 154,678 weights predicts, for every image column, the probability of a groove and its depth.

Results. On the sets that look like its training data it does well (`ideal` MAE 0.100 mm, `stress` 0.483 mm, 2 ms per frame). Then it fails in every way that matters for a measuring instrument (Figure 10):

- On another holder unit (`shifted`) the MAE is 5.2 mm with 8.7 % unsafe PASS. A calibration cannot fix it, because the network has no calibration input.
- With the laser switched *off* it still "measures" 52–91 % of the grooves. It learned that grooves look dark, and that darker means deeper in the simulator.
- On 67 real photos that cannot be measured it reported a depth on 43, and never said FAIL.
- At the legal boundary it is the least safe method tested (2.7 % unsafe PASS, against 0 for the baseline).

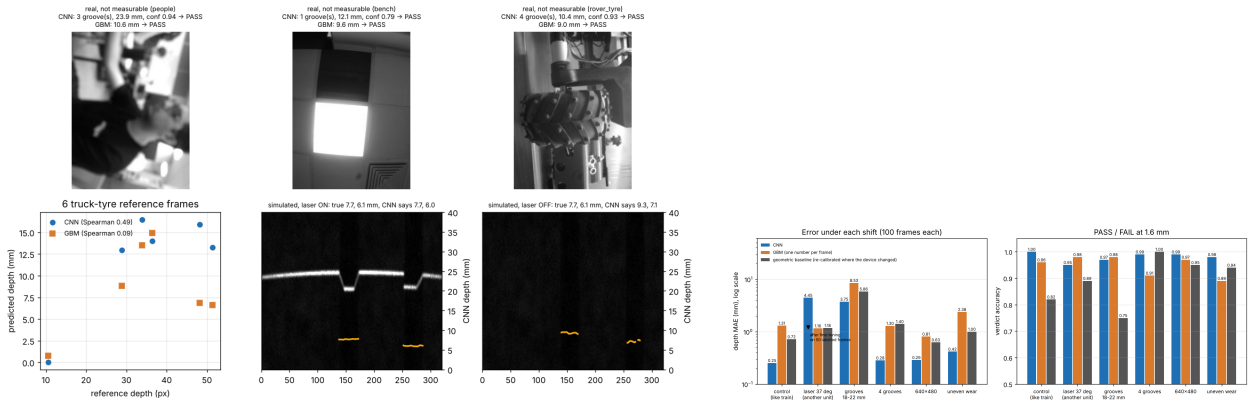


Figure 10: Left: the end-to-end network reports confident depths on photos of people and a ceiling light, and still finds grooves with the laser off. Right: its error under changes it was not trained for, against the geometric baseline.

Takeaway. For a safety measurement, keep the geometry as the measurement. Triangulation is correct for a reason we can explain to an auditor; a network's answer is only correct while the pictures look like its training set. Use learning as a helper (gate, line finder, cross-check), not as the ruler.